

Лабораторна робота №3

Створення та моделювання баз даних в MongoDB

1.1. Мета лабораторної роботи

Ознайомлення з роботою MongoDB у консолі. Отримання навичок створення бази даних, колекцій і документів, їх вилучення та перейменування.

1.2. Інструкція до виконання завдання

Робота у середовищі MongoDB може виконуватись кількома способами:

- через архітектуру «клієнт-сервер» (сервер **mongod** і клієнтський інтерфейс **mongosh**, або графічний клієнт **MongoDB Compass**);
- через хмару (**MongoDB Atlas Cluster**, що рекомендується для сучасних проєктів).

У межах цього курсу ознайомимось з роботою у середовищі MongoDB через архітектуру «клієнт-сервер».

Запуск MongoDB

- 1) Завантажте MongoDB Community Server.
- 2) Встановіть MongoDB.
- 3) Встановіть MongoDB Shell (mongosh).

На Windows/Linux/macOS серверна частина (**mongod**) зазвичай встановлюється як сервіс і запускається автоматично.

Шлях до даних задається у конфігураційному файлі **mongod.cfg** (Windows) або **mongod.conf** (Linux/macOS).

Для підключення до сервера відкрийте термінал і введіть: **mongosh**.

У відповідь ви отримаєте інформацію про версію MongoDB та підключення до **localhost:27017** (порт за замовчуванням).

```
mongosh mongodb://127.0.0.1
Microsoft Windows [Version 10.0.26100.6584]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\System32>mongosh
Current Mongosh Log ID: 68d2ea14a97f5c873ccea3
Connecting to:  mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.5.8
Using MongoDB:  8.2.0
Using Mongosh:  2.5.8

For mongosh info see: https://www.mongodb.com/docs/mongosh-shell/

To help improve our products, anonymous usage data is collected and sent to MongoDB periodically (https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

-----
The server generated these startup warnings when booting
2025-09-23T21:41:44.392+03:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
-----

test> |
```

Для прикладу виведемо список наявних баз даних **show dbs**

```
mongosh mongodb://127.0.0.1
test> show dbs
admin      40.00 KiB
config     108.00 KiB
local      64.00 KiB
test> |
```

Виведені на екран БД створені MongoDB автоматично.

Для того, щоб переключитися на іншу базу даних, необхідно виконати команду: **use** та написати назву необхідної БД, наприклад **use video**. Якщо такої БД не було серед списку можливих, то вона створиться сама автоматично:

```
mongosh mongodb://127.0.0.1
test> use video
switched to db video
video> |
```

Тепер ознайомимось із базовими командами роботи із СКБД MongoDB у консолі.

Довідка	
help	Загальна довідка
db.help()	Методи бази даних
db.mycoll.help()	Методи колекції
sh.help()	Команди шардингу
rs.help()	Команди реплікації
help admin	Довідка з адміністрування
help connect	Довідка з підключення
help keys	Список гарячих клавіш
Робота з базами та колекціями	
show dbs	Показати всі бази даних
use <db_name>	Перемикання на БД (створює, якщо не існує)
show collections	Список колекцій у поточній БД
show users	Користувачі поточної БД
db.createCollection("name")	Створити колекцію
db.collection.renameCollection("new_name")	Перейменувати колекцію
db.collection.drop()	Видалити колекцію
db.dropDatabase()	Видалити поточну БД
CRUD-операції	
db.collection.insertOne({ ... })	Вставити один документ
db.collection.insertMany([{ ... }, { ... }])	Вставити кілька документів
db.collection.find()	Знайти документи
db.collection.find().pretty()	Форматований вивід результату
db.collection.deleteOne({ ... })	Видалити один документ
db.collection.deleteMany({ ... })	Видалити кілька документів
Вихід	
exit	Вийти з MongoDB Shell

Робота з колекціями

Створення колекцій

Наповнимо базу даних «**video**» колекціями. Наприклад, пропонуємо створити колекцію «**cartoons**», де будуть зберігатись документи про мультфільми.

Прототип команди такий:

```
db.createCollection(<name>, {  
  capped: <boolean>,  
  size: <number>,  
  max: <number>,  
  storageEngine: <document>,  
  validator: <document>,  
  validationLevel: <string>,  
  validationAction: <string>,  
  indexOptionDefaults: <document>,  
  viewOn: <string>,  
  pipeline: <pipeline>,  
  collation: <document>,  
  writeConcern: <document>  
})
```

Більше про значення кожного із полів команди можна почитати на офіційному сайті MongoDB. У більшості випадків дана команда використовується у СКБД MongoDB для створення обмеженої колекції.

Приклад створення обмеженої колекції:

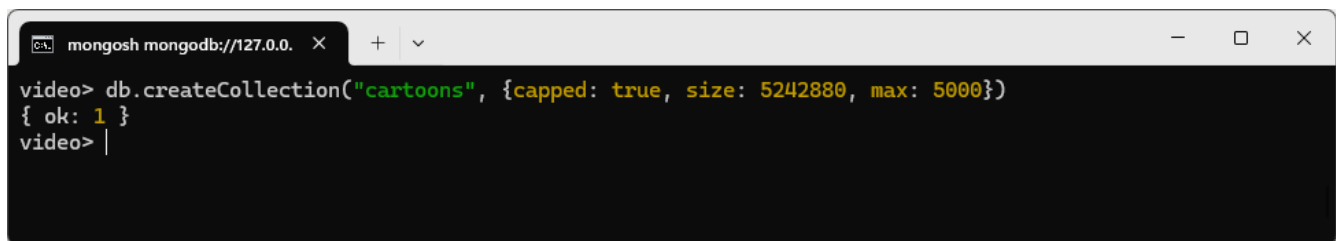
Наприклад, створимо колекцію «cartoons» із такими параметрами:

- **capped: true** — означає, що колекція створюється з обмеженнями (в іншому випадку, треба вказувати замість «true», «false» чи взагалі це поле пропустити);
- **size: 5242880** — визначає максимальне обмеження розміру колекції у байтах. Після досягнення максимального розміру колекції СКБД MongoDB видаляє старі документи, щоб звільнити місце для нових документів;
- **max: 5000** — визначає максимальну кількість документів у колекції. Обмеження розміру (поле «size») має перевагу над цим обмеженням (поле «max»).

Тобто, якщо розмір колекції перевищує 5242880 байт (або 5 МБ), тоді додаткові документи створюються тільки через видалення старих, навіть, якщо їх кількість є меншою за 5000.

Отже, для створення колекцію «**cartoons**» із заданими налаштуваннями необхідно виконати команду:

```
db.createCollection("cartoons",{
  capped: true,
  size: 5242880, // 5MB
  max: 5000 // максимум документів
})
```

A screenshot of a terminal window titled 'mongosh mongod://127.0.0.1'. The prompt is 'video>'. The user enters the command 'db.createCollection("cartoons", {capped: true, size: 5242880, max: 5000})'. The terminal returns the response '{ ok: 1 }'. The prompt then changes to 'video> |'.

Система зреагувала написом { "ok" : 1 }, що означає, що у підключеній БД створено одну колекцію.

Можемо перевірити факт створення колекції через команду **show collections**

A screenshot of a terminal window titled 'mongosh mongod://127.0.0.1'. The prompt is 'video>'. The user enters the command 'show collections'. The terminal returns the response 'cartoons'. The prompt then changes to 'video> |'.

Робота з документами

Вставка документів

Тепер створимо у цій колекції документ (запис). Це можна зробити за допомогою команд: **insertOne()** та **insertMany()**.

```
// Вставка одного документа
db.cartoons.insertOne({
  "title": "Tom and Jerry",
  "year": 1940,
  "genre": ["comedy", "animation"],
  "country": "USA",
  "type": "serial",
  "episodes": 164,
  "created_at": new Date()
})
```

```
mongosh mongodb://127.0.0.1:27027 > use video
video> db.cartoons.insertOne({
...   "title": "Tom and Jerry",
...   "year": 1940,
...   "genre": ["comedy", "animation"],
...   "country": "USA",
...   "type": "serial",
...   "episodes": 164,
...   "created_at": new Date()
... })
{
  acknowledged: true,
  insertedId: ObjectId('68d2f5dda97f5c873ccebea4')
}
video> |
```

Таким чином, до колекції «**cartoons**» бази даних «**video**» додано документ із записом про мультфільм «**Tom and Jerry**».

Запис «**acknowledged: true**» означає, що запит було успішно виконано, а поле «**insertedId**» показує який унікальний ідентифікатор було присвоєно даному документу.

Кожен документ в базі даних повинен мати поле «**_id**», яке має унікальне значення. Якщо під час створення нового запису поле «**_id**» не було вказано, то MongoDB автоматично присвоїть унікальний номер.

За допомогою команди **insertOne()** та **insertMany()** можна також створювати нові колекції.

// Вставка кількох документів

```
db.movies.insertMany([
  {
    "title": "The Matrix",
    "year": 1999,
    "genre": ["action", "sci-fi"],
    "rating": 8.7,
    "director": "Wachowski Sisters"
  },
  {
    "title": "Inception",
    "year": 2010,
    "genre": ["action", "sci-fi", "thriller"],
    "rating": 8.8,
    "director": "Christopher Nolan"
  }
])
```



```
mongosh mongodb://127.0.0.1:27027/video> db.movies.find()
[
  {
    _id: ObjectId('68d2f88da97f5c873ccea5'),
    title: 'The Matrix',
    year: 1999,
    genre: [ 'action', 'sci-fi' ],
    rating: 8.7,
    director: 'Wachowski Sisters'
  },
  {
    _id: ObjectId('68d2f88da97f5c873ccea6'),
    title: 'Inception',
    year: 2010,
    genre: [ 'action', 'sci-fi', 'thriller' ],
    rating: 8.8,
    director: 'Christopher Nolan'
  }
]
video> |
```

У **mongosh** метод **.pretty()** більше не потрібен - форматування відбувається автоматично.

Звернемо увагу, що окрім інформації, яку ми вводили, з'явилось ще одне поле «**_id**», яке має значення виду **ObjectId** («унікальний_номер») За допомогою запиту «**find**» можна знаходити не тільки колекції, але і поля у них. Наприклад, знайдемо у колекції «**cartoons**» бази даних «**video**» запис із назвою «**Tom and Jerry**»:

db.cartoons.find({"title": "Tom and Jerry"})

```
mongosh mongodb://127.0.0.1:27027/video> db.cartoons.find({"title": "Tom and Jerry"})
[
  {
    _id: ObjectId('68d2f5dda97f5c873ccea4'),
    title: 'Tom and Jerry',
    year: 1940,
    genre: [ 'comedy', 'animation' ],
    country: 'USA',
    type: 'serial',
    episodes: 164,
    created_at: ISODate('2025-09-23T19:32:45.973Z')
  }
]
video> |
```

Знайдемо у колекції «**cartoons**» бази даних «**video**» запис із назвою «**Ice Age**»:

db.cartoons.find({"title": "Ice Age"})

```
mongosh mongodb://127.0.0.1:27021/
video> db.cartoons.find({"title": "Ice Age"})
video> |
```

СКБД MongoDB команду виконала, але не вивела на екран цей документ, бо його в колекції «**cartoons**» не існує.

Приклад інших команд:

// Всі документи в колекції

```
db.movies.find()
```

// Пошук з умовами

```
db.movies.find({"year": 1999})
```

// Пошук з операторами порівняння

```
db.movies.find({"year": {$gte: 2000}})
```

// Пошук одного документа

```
db.movies.findOne({"title": "The Matrix"})
```

// Підрахунок документів

```
db.movies.countDocuments()
```

// Пошук з проєкцією (вибірка полів)

```
db.movies.find({}, {"title": 1, "year": 1, "_id": 0})
```

// Сортування

```
db.movies.find().sort({"rating": -1})
```

// Обмеження кількості результатів

```
db.movies.find().limit(5)
```

Оновлення документів

// Оновлення одного документа

```
db.movies.updateOne(
  {"title": "The Matrix"},
  {$set: {"rating": 8.8, "updated": new Date()}}
)
```

// Оновлення кількох документів

```
db.movies.updateMany(
  {"year": {$lt: 2000}},
  {$set: {"category": "classic"}}
)
```

```
// Заміна документа
db.movies.replaceOne(
  {"title": "The Matrix"},
  {
    "title": "The Matrix",
    "year": 1999,
    "rating": 8.8,
    "director": "Wachowski Sisters",
    "remastered": true
  }
)
```

Видалення документів

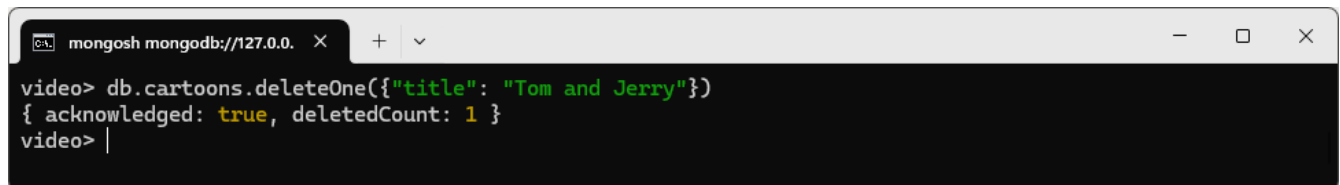
```
// Видалення одного документа
db.movies.deleteOne({"title": "The Matrix"})

// Видалення кількох документів
db.movies.deleteMany({"year": {$lt: 1980}})

// Видалення всіх документів з колекції
db.movies.deleteMany({})
```

Припустимо, що документ про мультфільм «**Tom and Jerry**» у колекції «**cartoons**» було додано випадково і його треба видалити.

```
db.cartoons.deleteOne({"title": "Tom and Jerry"})
```

A screenshot of a terminal window titled 'mongosh mongod://127.0.0.1'. The prompt is 'video>'. The command entered is 'db.cartoons.deleteOne({"title": "Tom and Jerry"})'. The output is '{ acknowledged: true, deletedCount: 1 }'. The prompt is now 'video> |'.

Звернемо увагу, що у СКБД MongoDB з обмеженої (capped) колекції можна видаляти документи за допомогою deleteOne() та deleteMany().

Керування колекціями

```
// Перевірка чи є колекція обмеженою
db.cartoons.isCapped()

// Переіменування колекції
db.movies.renameCollection("films")

// Видалення колекції
db.films.drop()

// Видалення бази даних
db.dropDatabase()
```

1.3. Завдання роботи

1. Ознайомитися із поняттями «база даних», «колекція» та «документ» у MongoDB та порівняти з реляційними СУБД.
2. Ознайомитися з інструкцією до виконання лабораторної роботи.
3. Ознайомитися із базовими командами роботи з MongoDB у консолі.
4. Обрати тему для циклу лабораторних робіт (пропоновані теми подано нижче).
5. Створити БД і дві колекції, заповнивши їх до 10 документами (кожен документ містить 4–5 полів різних типів).
6. Виконати практичні завдання:
 - Створення та вилучення БД.
 - Створення, перейменування та вилучення колекцій.
 - Створення, пошук документів та внесення змін до документів.
7. Відпрацювати моделювання зв'язків між документами (у межах однієї або кількох колекцій):
 - Embedded Documents (вкладені документи).
 - References (посилання через ObjectId).
 - \$lookup (агрегація-з'єднання).

Для кожного типу створити щонайменше 5 документів і виконати 2–3 приклади запитів (пошук документів).
8. Відповісти на контрольні питання.
9. Оформити звіт з лабораторної роботи.

Рекомендовані теми:

- | | |
|--------------------------------|--|
| ✓ Агентство нерухомості | ✓ Приватна клініка |
| ✓ Бібліотека | ✓ Видавництво |
| ✓ Комісійний автомагазин | ✓ Магазин комп'ютерної техніки |
| ✓ Футбольний чемпіонат | ✓ Кафедра ВУЗу |
| ✓ Агентство з працевлаштування | ✓ Академічна успішність |
| ✓ Рекламне агентство | ✓ Інтернет-магазин |
| ✓ Аукціон | ✓ Медична система |
| ✓ Адвокатура | ✓ Система управління проектами |
| ✓ Фірма по зборі ПК | ✓ Ресторан (меню, замовлення, клієнти) |

Деталі до завдання з моделювання зв'язків у MongoDB

У рамках лабораторної роботи необхідно відпрацювати моделювання зв'язків між документами в колекціях. Для цього слід обрати та реалізувати щонайменше 2–3 типи зв'язків:

1. **Embedded Documents (вкладені документи)** – зберігання пов'язаних даних у вигляді піддокументів всередині основного документа.

Приклад: у документі про книгу зберігати інформацію про автора як піддокумент.

2. **References (посилання через ObjectId)** – зберігання посилань на інші документи через унікальний ідентифікатор `_id`.

Приклад: у документі про замовлення зберігати лише `customerId`, який посилається на окремий документ у колекції клієнтів.

3. **\$lookup (агрегація-з'єднання)** – використання агрегування для об'єднання даних з різних колекцій за спільним ключем.

Приклад: з'єднати колекцію «orders» з колекцією «customers», щоб отримати повну інформацію про замовлення разом із даними клієнта.

1.4. Дайте відповідь на питання

1. Що таке у MongoDB «колекція»?
2. У чому суть «обмеженої колекції» (capped collection)?
3. Поясніть, що відбудеться під час виконання у консолі такої команди:
db.createCollection("profile", {capped: true, size: 9500, max: 150})
4. Для яких цілей можна використовувати команду **db.collection.isCapped()**?
5. Який синтаксис команди для відображення усіх створених колекцій у межах поточної БД?
6. Поясніть команду:
db.cartoons.insertOne({"title": "Tom and Jerry", "year": 1940})
7. У чому різниця між **mongosh** та застарілим **mongo shell**?
8. Яка різниця між командами **insertOne()** та **insertMany()**?
9. Для чого потрібні команди **deleteOne()** та **deleteMany()**?
10. Як можна оновити документи?
11. Під час створення нової обмеженої колекції за допомогою команди **db.createCollection()**, який параметр має пріоритет — **size** чи **max**?
12. За допомогою якої команди можна видалити колекцію: **db.collection.drop()** чи **db.collection.deleteMany({})**? У чому принципова різниця?
13. Що таке **Schema Validation** у MongoDB і як вона допомагає забезпечити цілісність даних?
14. Які переваги має **MongoDB Atlas** порівняно з локальною установкою для навчання?
15. Як змоделювати зв'язок "один до багатьох" всередині одного документа MongoDB?
16. У чому різниця між Embedded Documents та References під час моделювання зв'язків у MongoDB?
17. Які основні типи даних підтримує BSON у MongoDB?
18. Чим відрізняються типи даних ObjectId, Date, Array, Embedded Document?
19. Які особливості зберігання чисел у BSON (int32, int64, double, decimal128)?

1.5. Література до лабораторної роботи

1. **MongoDB** : What is MongoDB? [Електронний ресурс] // Офіційний портал MongoDB Inc. — Режим доступу до ресурсу: <https://www.mongodb.com/what-is-mongodb>.
2. **MongoDB**: дані Вікіпедії [Електронний ресурс] // Wikipedia. — Режим доступу до ресурсу: <https://uk.wikipedia.org/wiki/MongoDB>.
3. **Install MongoDB** [Електронний ресурс] // Офіційний портал MongoDB Inc. — Режим доступу до ресурсу: <https://www.mongodb.com/docs/manual/installation/>.
4. **MongoDB Shell** [Електронний ресурс] // Офіційний портал MongoDB Inc. — Режим доступу до ресурсу: <https://www.mongodb.com/try/download/shell>.
5. **db.createCollection()** [Електронний ресурс] // Офіційний портал MongoDB Inc. — Режим доступу: <https://www.mongodb.com/docs/manual/reference/method/db.createCollection/>